

Musterlösung — Übungsklausur 3

Deep Learning 1 · TU Berlin

1 Multiple Choice

1. Stateful vs. Stateless?

✓ (b) Stateful nutzt neben Input auch internen Zustand für Vorhersage

- (a) Stateless deterministisch
- (c) Stateless mehr Parameter
- (d) Stateful keine Sequenzen

2. Softmax für Klasse i?

✓ (b) $p_i = \exp(y_i) / \sum_j \exp(y_j)$

- (a) Division ohne exp
- (c) sigmoid
- (d) log

3. Zentrales Problem bei LRP?

✓ (b) Propagationsstrategie muss für jede neue Architektur angepasst werden

- (a) Nur lineare Modelle
- (c) Negative Relevanz
- (d) Doppelte Forward Passes

4. Activation Maximization?

✓ (b) Finden eines Inputs, der einen bestimmten Neuron maximal aktiviert

- (a) Maximale Lernrate
- (c) Loss maximieren
- (d) Schicht mit max. Aktivierung

5. MLE mit Laplace-Verteilung → welcher Loss?

✓ (c) Absolute Loss $|y-t|$

- (a) Squared
- (b) Log-Cosh
- (d) Cross-Entropy

6. Wann stoppt Perceptron?

✓ (b) Sobald alle Trainingsbeispiele korrekt klassifiziert sind

- (a) Feste Epochenzahl
- (c) Min. Validierungsfehler
- (d) Gradient null

7. Encoder-Decoder RNN?

✓ (a) Eingangssequenz → globale Repräsentation s → Ausgangssequenz aus s generieren

- (b) Parallel
- (c) Bidirektional
- (d) Kein Hidden State

8. Dataset Bias?

✓ (a) Überrepräsentation bestimmter Verteilungsbereiche → falsche Annahmen über Gesamtverteilung

- (b) Zu kleiner Datensatz
- (c) Fehlerhafte Labels
- (d) Falsche Lernrate

2 Theorie — Gradientenberechnung für Radiales Netz

(a) $\partial f / \partial x_i$

Kettenregel: $f = \sum_j w_j \cdot a_j$, $a_j = \exp(-\sum_i z_{ij})$, $z_{ij} = |x_i - c_{ij}|$

$$\partial f / \partial x_i = \sum_j w_j \cdot (\partial a_j / \partial x_i)$$

$$\partial a_j / \partial x_i = a_j \cdot (-\partial z_{ij} / \partial x_i) = a_j \cdot (-\text{sign}(x_i - c_{ij}))$$

$$\partial f / \partial x_i = -\sum_j w_j \cdot a_j \cdot \text{sign}(x_i - c_{ij})$$

(b) Gradientennorm-Schranke

$$\|\partial f / \partial x\| = \sum_i |\partial f / \partial x_i|$$

$$= \sum_i |-\sum_j w_j \cdot a_j \cdot \text{sign}(x_i - c_{ij})|$$

$$\leq \sum_i \sum_j |w_j| \cdot |a_j| \cdot |\text{sign}(\dots)| = \sum_i \sum_j |w_j| \cdot a_j$$

$$= d \cdot \sum_j |w_j| \cdot a_j$$

Da $a_j = \exp(-\dots) \in (0, 1]$ gilt: $\sum_j |w_j| \cdot a_j \leq \sum_j |w_j| = \|w\|$

$$\text{Also: } \|\partial f / \partial x\| \leq d \cdot \|w\|$$

(c) Anwendung der Schranke

Um kleine Gradientennorm zu garantieren: Regularisiere die Gewichte w mit einer L1- oder L2-Penalty, die $\|w\|$ klein hält. Damit wird die obere Schranke $d \cdot \|w\|$ klein, was lokale Robustheit des Modells gegenüber Inputperturbationen gewährleistet.

3 Theorie — Cross-Entropy und Log-Loss

(a) Cross-Entropy für C Klassen

$$H(y, t) = H(q, p) = -\sum_{i=1}^C q_i \cdot \log(p_i)$$

$$p_i = \text{softmax}(y)_i = \exp(y_i) / \sum_j \exp(y_j)$$

Softmax konvertiert rohe Scores y in normierte Wahrscheinlichkeiten. Cross-Entropy misst KL-Divergenz zwischen Zielverteilung q (One-Hot) und Modellverteilung p .

(b) Log-Loss $\equiv$ Cross-Entropy (binär)

Gegeben: $p = (\exp(-y)/(1+\exp(-y)), \exp(y)/(1+\exp(y)))$, $q = (1_{\{t<0\}}, 1_{\{t>0\}})$

$$H(q, p) = -1_{\{t<0\}} \cdot \log(\exp(-y)/(1+\exp(-y))) -$$

$$1_{\{t>0\}} \cdot \log(\exp(y)/(1+\exp(y)))$$

$$= -\log(\exp(yt)/(1+\exp(yt)))$$

$$= -\log(1/(1+\exp(-yt)))$$

$$= \log(1 + \exp(-yt))$$

(c) Log-Loss vs. Perceptron Loss

Perceptron Loss: 0 für korrekte Punkte — Gradient verschwindet, Training stoppt sobald alle korrekt klassifiziert. Keine Margin-Optimierung.

Log-Loss: Penalisiert auch korrekte Punkte die zu nah an der Grenze sind. Schiebt Entscheidungsgrenze von Trainingsdaten weg $\rightarrow$ wirkt als impliziter Regularisierer $\rightarrow$ bessere Generalisierung.

4 Theorie — Mixture Density Networks

(a) Inverses Problem

Ein inverses Problem ist eine Aufgabe, bei der mehrere Input-Konfigurationen zum gleichen Output führen — d.h. die Abbildung ist nicht injektiv. Ein einfaches Netz mit Squared Loss würde den Durchschnitt aller Lösungen vorhersagen (unphysikalisch).

Beispiel: Roboterarm-Kinematik (inverse Kinematik) — viele Gelenkwinkelkonfigurationen führen zur gleichen Endposition.

(b) MDN Formel

$$p(t|x) = \sum_{i=1}^M \alpha_i(x) \cdot \phi_i(t|x)$$

$$\phi_i(t|x) = N(t; \mu_i(x), \sigma_i(x)^2) \text{ [Gaußkomponente]}$$

α_i : Mischungsgewichte (Softmax), μ_i : Mittelwerte, σ_i : Standardabweichungen.

(c) MDN Ausgabevektor

$$z = \{\alpha_i, \mu_i, \sigma_i\}_{i=1}^M$$

Aktivierungen: $\alpha_i \rightarrow \text{Softmax}$ (Normierung auf 1). $\sigma_i \rightarrow \exp$ (Positivität). $\mu_i \rightarrow$ keine Aktivierung.

Beispiel Planeten: Forward Map: Zusammensetzung $\rightarrow$ Masse, Radius. MDN lernt inverse Map: aus Masse+Radius $\rightarrow$ Verteilung über mögliche Zusammensetzungen.

5 Theorie — Sparse Coding

(a) Optimierungsziel mit L1 + Gewichtsregularisierung

$$\min_{\{W, s_1, \dots, s_N\}} (1/N) \sum_i [\|x_i - W \cdot s_i\|^2 + \lambda \|s_i\|_1] + \eta \cdot \|W\|_F^2$$

Term 1: Rekonstruktionsfehler. Term 2: L1-Sparsitätsstrafe auf Codes. Term 3:

Frobenius-Regularisierung auf Dictionary.

(b) Problem ohne Gewichtsregularisierung

Ohne Term 3: W kann beliebig skaliert werden. Skaliere $W \rightarrow \epsilon \cdot W$, $s \rightarrow s/\epsilon$: Rekonstruktionsfehler bleibt gleich, $\|s\|_1 \rightarrow \|s\|_1/\epsilon \rightarrow 0$ (beliebig klein). L1-Strafe wird wirkungslos, kein echter Sparsity-Druck.

(c) Online vs. Batch

Batch (kleine N): Aktualisiert W und alle sparse Codes in einer Iteration. Exakter, aber teuer für großes N .

Online (großes N): Wählt zufälliges Sample, inferiert s_i , aktualisiert W approximativ. Skaliert gut, für große Datensätze bevorzugt.